



Serviço Nacional de Aprendizagem Industrial

PELO FUTURO DO TRABALHO

Introdução a Classes e Objetos

Professor: Henrique Delegrego

Introdução a Classes e Objetos

O que são classes e objetos

- Classes são estruturas ou moldes que definem a forma e comportamento de um objeto
- Um objeto é uma instância de uma classe com estado e comportamento específico

Exemplo de Classe e Objeto

Classe Veiculo:

- Marca
- Modelo
- Placa
- Ano
- Preço

(Atributos de classe)



Objeto Civic:

- Marca = "Honda"
- Modelo = "Civic"
- Placa = "AAA1A11"
- Ano = 2010
- Preço = 50000

Introdução a Classes e Objetos

Classe no Java

Classe Veiculo:

- Marca
- Modelo
- Placa
- Ano
- Preço

```
// A classe representa um tipo de objeto
// Neste caso, estamos criando um modelo para representar um veículo
public class Veiculo {

    // Atributos de classe: características que um objeto de Veiculo possui

    String marca;

    String modelo;

    String placa;

    int ano;

    double preco;
}
```

Introdução a Classes e Objetos

Instanciação de um objeto

```
// Classe principal do programa que contém o método main
public class ClassePrincipal {

    public static void main(String[] args) {

        // Cria um novo objeto da classe Veiculo
        // O objeto é armazenado na variável chamada "civic"
        Veiculo civic = new Veiculo();

        Veiculo sandero = new Veiculo();

        Veiculo onix = new Veiculo();

    }
}
```



Instanciação de um objeto

Introdução a Classes e Objetos

```
public class ClassePrincipal {  
  
    public static void main(String[] args) {  
  
        Veiculo civic = new Veiculo();  
  
        // Define a marca do veículo.  
        // Estamos acessando o atributo "marca" do objeto "civic"  
        civic.marca = "Honda";  
        civic.modelo = "Civic";  
        civic.placa = "XXX1X11";  
        civic.ano = 2010;  
        civic.preco = 50000;  
  
        // Exibe informações do veículo  
        System.out.println(civic.marca + " " + civic.modelo + " à venda por somente R$" + civic.preco);  
  
    }  
}
```

Honda Civic à venda por somente R\$50000.0

Agora:

- Vamos implementar a classe Veiculo
- Façam o exercício 1 a 3

Introdução a Classes e Objetos

Métodos em classes

- Vão definir os comportamentos / funções de um objeto

```
3 public class Quadrilatero {  
4  
5     double altura;  
6     double largura;  
7  
8     public double obterArea() {  
9         return altura * largura;  
10    }  
11 }
```

```
3 public class ClassePrincipal {  
4  
5     public static void main(String[] args) {  
6  
7         Quadrilatero q1 = new Quadrilatero();  
8         q1.altura = 2;  
9         q1.largura = 4;  
10  
11        System.out.println(q1.obterArea());  
12    }  
13 }
```

Agora:

- Façam os exercícios 4 a 6

Introdução a Classes e Objetos

Método construtor

- Método especial usado para inicializar um novo objeto quando ele é criado, configurando o estado inicial de um objeto, atribuindo valores às suas variáveis de instância e realizando qualquer configuração necessária
- Invocado automaticamente quando usada a palavra-chave **new** para criar um objeto
- Tem o mesmo nome da classe e não possui um tipo de retorno
- Se uma classe não define nenhum construtor, o Java fornece automaticamente um construtor padrão
- Uma classe pode ter múltiplos construtores, desde que suas listas de parâmetros sejam diferentes

Introdução a Classes e Objetos

Método construtor

```
public class Veiculo {  
  
    // Atributos de classe  
    String marca;  
    String modelo;  
    String placa;  
    int ano;  
    double preco;  
  
    // Construtor da classe Veiculo  
    // Executado automaticamente quando usamos "new Veiculo()"  
    // Recebe os valores que serão utilizados para inicializar os atributos do objeto  
    public Veiculo(String marca, String modelo, String placa, int ano, double preco) {  
        this.marca = marca;  
        this.modelo = modelo;  
        this.placa = placa;  
        this.ano = ano;  
        this.preco = preco;  
    }  
}
```

- A palavra-chave **this** é uma referência ao próprio objeto da classe onde é usado. Utilizado dentro de métodos e construtores para se referir à instância atual do objeto, ou seja, ao objeto que invocou o método ou construtor

Introdução a Classes e Objetos

Método construtor

```
Veiculo civic = new Veiculo();
```



```
Veiculo civic = new Veiculo("Honda", "Civic", "AAA1A11", 2010, 50000);
```

Agora:

- Vamos criar o nosso método construtor

Introdução a Classes e Objetos

Método toString

- Quando você cria um objeto em Java e tenta exibi-lo diretamente, ele não gera uma saída legível por padrão
- Fornece uma representação em formato de texto de um objeto
- Criaremos um método na classe do modelo para imprimir os conteúdos dele
- Será necessário sobrescrever o método toString já existente por padrão

```
// Na classe Veiculo
@Override
public String toString() {
    return "Veículo: " + marca + " " + modelo + " (" + placa + ")\nAno: " + ano + "\nPreço: R$ " + preco;
}
```

Introdução a Classes e Objetos

Método toString

```
Veiculo civic = new Veiculo("Honda", "Civic", "AAA1A11", 2010, 50000);  
  
System.out.println(civic);
```

Saída sem o método toString:

Veiculo@1dbd16a6

Saída com o método toString:

Veículo: Honda Civic (AAA1A11)
Ano: 2010
Preço: R\$ 50000.0

Agora:

- Façam o exercício 7

Introdução a Classes e Objetos

Encapsulamento

- Usado para limitar o acesso a atributos e métodos
- Também permite esconder detalhes internos de implementação
- Pode ser por motivos de segurança ou por motivos de validação de valores

```
public class Veiculo {  
  
    // Atributos encapsulados  
    private String marca;  
    private String modelo;  
    private String placa;  
    private int ano;  
    private double preco;  
  
    // "private" significa que esses atributos só podem ser acessados diretamente dentro da classe Veiculo  
  
}
```

Introdução a Classes e Objetos

Métodos get e set

- Fornecem uma maneira controlada de acessar os atributos de uma classe
- Necessários após encapsular os atributos

Getter:

- Um getter é um método que “pega” o valor de um atributo privado de uma classe
- Normalmente não recebe parâmetro

Setter:

- Um setter é um método que define o valor de um atributo privado da classe
- Tem como parâmetro o novo valor do atributo

Agora:

- Vamos fazer os métodos get e set

Introdução a Classes e Objetos

Métodos get e set

- Podem ser colocadas validações nos métodos **set**

```
// Permite alterar a altura do objeto, mas antes verifica
// se o valor recebido é válido.
public void setAltura(double altura) {

    // Verifica se a altura é menor ou igual a zero
    if (altura <= 0) {

        // Informa ao usuário que o valor fornecido não é válido
        System.out.println("Erro, altura inválida");

    } else {

        // Se a altura for válida, o valor recebido pelo parâmetro
        // é armazenado no atributo "altura" do objeto.
        // "this.altura" = atributo do objeto
        // "altura" = parâmetro do método.
        this.altura = altura;
    }
}
```

Introdução a Classes e Objetos

Métodos get e set

- O construtor **deve** ser modificado para fazer atribuições pelos sets

```
public Quadrilatero(double altura, double largura) {  
    // Em vez de alterar o atributo diretamente com "this.altura = altura"  
    // usamos o setter para aplicar as regras de validação na criação do objeto  
    setAltura(altura);  
    setLargura(largura);  
}
```

Agora:

- Façam o exercício 8

Introdução a Classes e Objetos

Tratamento de Exceção

- Uma **exceção** é um evento inesperado que interrompe o fluxo normal do programa
- Pode ser causada por diversos fatores, como entrada inválida do usuário, arquivos não encontrados, divisão por zero ou problemas de conexão

```
int vetor[] = new int[5];
```

Exceção != Erro

```
vetor[5] = 20;
```

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 5  
    at Exemplo1.main(Exemplo1.java:8)
```

Introdução a Classes e Objetos

Tratamento de Exceção

- Nomenclaturas do Java:
 - Uma exceção é lançada (**throw**)
 - Podemos pegar ela (**catch**) para que o programa não pare
- Para tratar exceções usamos:
 - Bloco de try catch
 - throw

Introdução a Classes e Objetos

Bloco try-catch

- O bloco **try-catch** permite que o programa lide com erros de forma controlada, sem falhar abruptamente
- O bloco **try** contém o código que pode gerar uma exceção, enquanto o **catch** captura e trata essa exceção caso ocorra
- Se uma exceção for lançada dentro do **try**, o fluxo do programa será transferido para o **catch**

```
6       Scanner input = new Scanner(System.in);
7
8       int vetor[] = new int[5];
9
10      try {
11          int i = input.nextInt();
12          vetor[5] = i;
13      } catch (Exception e) {
14          System.out.println(e);
15      }
16      System.out.println("O programa continuou");
17
18  }
```

Introdução a Classes e Objetos

Múltiplos blocos de catch

- Às vezes, um mesmo trecho de código pode gerar diferentes tipos de exceções, e queremos tratar cada uma de maneira distinta
- Para isso, se usa um único bloco try seguido de vários blocos catch

```
8      Scanner input = new Scanner(System.in);
9
10     int vetor[] = new int[5];
11
12     try {
13         int valor = input.nextInt();
14         vetor[5] = valor;
15     } catch (InputMismatchException e) {
16         System.out.println("Tipo inválido");
17     } catch (ArrayIndexOutOfBoundsException e) {
18         System.out.println("Fora do limite");
19     }
20
21     System.out.println("O programa continuou...");
22
23 }
```

Introdução a Classes e Objetos

Finally

- O bloco **finally** sempre será executado, independentemente de ocorrer uma exceção ou não
- Utilizado para liberar recursos, como arquivos e conexões com o banco de dados

```
Scanner input = new Scanner(System.in);

int vetor[] = new int[5];

try {
    int valor = input.nextInt();
    vetor[5] = valor;
} catch (InputMismatchException e) {
    System.out.println("Tipo inválido");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Fora do limite");
} finally {
    // Tudo que está aqui vai rodar
    // Independente se aconteceu alguma exceção ou não
    // Usado para limpeza do código e legibilidade
    input.close(); // Por exemplo
}
```

Introdução a Classes e Objetos

Try-With-Resources

- Recurso que facilita o gerenciamento automático de objetos que precisam ser fechados após o uso, evitando vazamentos de recursos como conexões de banco de dados, sockets, streams e listeners
- Garante que cada recurso seja fechado automaticamente, sem que você precise escrever um bloco finally manualmente

```
// Try-with-resources: Scanner será fechado automaticamente
try (Scanner input = new Scanner(System.in)) {

    System.out.print("Insira seu nome: ");
    String nome = input.next();
    System.out.println("Olá, " + nome);

} catch (Exception e) {

    System.out.println("Ocorreu um erro: " + e.getMessage());
}
```

Introdução a Classes e Objetos

Lançamento de Exceção

- O comando **throw** é usado para lançar exceções explicitamente
- Diferente do **try-catch**, que serve para tratar exceções, o **throw** permite que o próprio código gere uma exceção quando uma situação inesperada ocorre

```
public void setPreco(double preco) {  
    if (preco <= 0) {  
        throw new IllegalArgumentException("Preço inválido");  
    }  
    this.preco = preco;  
}  
  
public void setNome(String nome) {  
    if (nome == null || nome.isBlank()) {  
        throw new IllegalArgumentException("Nome inválido");  
    }  
    this.nome = nome;  
}
```

Introdução a Classes e Objetos

Lançamento de Exceção

```
public static void main(String[] args) {  
  
    Usuario u = new Usuario("    ");  
  
}
```

```
Exception in thread "main" java.lang.IllegalArgumentException: Nome inválido  
    at Usuario.setNome(Usuario.java:17)  
    at Usuario.<init>(Usuario.java:9)  
    at ClassePrincipal.main(ClassePrincipal.java:8)
```

Agora:

- Terminem a lista de exercícios



Serviço Nacional de Aprendizagem Industrial

PELO FUTURO DO TRABALHO

0800 048 1212     **sc.senai.br**

Rodovia Admar Gonzaga, 2765 - Itacorubi - 88034-001 - Florianópolis, SC